

M6: 字符、字符数组与字符串

CSC3002 基础讲义 · 第 2 册 · 前置: M2 的参数与 const、M3-M5 的数组和指针

核心问题: 内存里到底保存了几个字符, 程序又凭什么知道文本在哪里结束? 学完应能区分 'a' 与 "a", 画出 `char text[] = "abc";` 的元素, 分清数组容量、C 字符串长度和 `std::string` 的长度, 并读懂 A1 的字符串接口。

主线: 单个 char → 字符字面量 → 字符数组 → 字符串字面量与 `\0` → 指针访问 → 长度和容量 → 复制、比较、输入 → `std::string` 的操作与应用 → A1。采用 C++17; 本册把 L4 和 L6 的对应例题放回主线, 并详细分析 E22 Q3。

阅读依据与课程定位

这是对实际课程材料重新核对后的修订版, 核对日期为 2026-10-08。L 后的数字是课件代号; “第几张” 按文件实际幻灯片顺序计数, 从封面开始, 不沿用旧模板页脚。E22 页码为 PDF 页码。

- L4: 4-CSC3002_Lecture Notes_Strings.pptx
- L6: 6-CSC3002_Pointers and Arrays(4).pptx
- E22: CSC3002 Final 2022 Spring.pdf
- A1: assignment1.pdf

本册学习任务	原材料定位	要能独立完成的动作
char、编码、字符分类	L4 第 5、8-9 张	分清字符值、整数和转义写法
C 字符串、容量、库函数与越界	L4 第 10-15 张; L6 第 39-44 张	画出零字符位置, 判断源和目的范围
string 拼接、方法和可修改性	L4 第 22-36 张; E22 Q3	从左到右检查每一个 + 的操作数类型
回文与首字母缩写应用	L4 第 37-38 张	把字符访问组合成一个小算法
固定容量字符串的行为要求	A1 第 1-2 页	辨认截断、独立副本与输入一词契约

课件决定本册的概念范围; 旧卷用于练题型和深度, 不预测今年题目或分值。本册的解释、状态表和答案由教学推导得到; 未找到官方答案册, 因此不称为官方标准答案。保留的最小例子与一般练习属于自编教学补充; 涉及原材料时逐项标明“课件例题”“原题代码”或“题意转述”。

当前 L1 第 2 张写的是闭卷, E22 第 1 页写的是可带一张双面 A4 的限制性开卷, 两者不能混作今年规则。六册只覆盖 C++ 基础这一段, 旧卷的数据结构、算法和部分继承多态内容仍需后续学习。

1. char 是什么? 它保存的是一个字符单位

char 是一种整数类型, 常用来保存普通字符编码中的一个代码单元。 在本册英文字符例子里, 可以先把它理解为一个字符位置。例如:

```
char ch = 'a';
```

char 是类型; ch 是对象名; 'a' 是普通单字符字面量, 类型为 char; 初始化让 ch 保存该字符的编码值。单引号是源代码写法的一部分, 不会和 a 一起存入 ch。

为什么需要 char? 要逐字符检查大小写、查找终止标记、修改文本，都需要从“整个字符串”下降到单个元素。`sizeof(char) == 1`，但 C++ 的一个字节不应无条件等同于八位；常见机器是八位，题目没给条件时要说明假设。

中文字符可能由多个代码单元表示，不能把“一个 char 一定等于一个人眼看到的字符”推广到所有文本。本册先研究普通英文和数字字符。

2. 字符不是“画出来的数字”：'5' 和 5 不同

```
char digit = '5';
int number = 5;
```

`digit` 保存数字字符的编码，`number` 保存数值 5。`std::cout << digit`；按字符输出，显示 5；`std::cout << number`；按整数输出，也显示 5。**显示相同，不代表类型和存储含义相同。**

普通数字字符的编码是连续的，因此在确认 `ch` 是 '0' 到 '9' 后，`ch - '0'` 可以取得对应数值：'5' - '0' 得到 5。相反，'0' + 5 先产生整数运算结果，再可转换为对应字符。

若题目给出 ASCII 编码，'A' 的值为 65，'a' 为 97。但不能把所有字符编码规则都当作 C++ 对任何环境的无条件保证。

3. 'a' 和 "a" 为什么完全不是同一种东西？

普通字符字面量 'a' 表示一个 char 值；普通字符串字面量 "a" 对应一个含结束零字符的 const char 数组。

写法	类型或结构	元素数量	用途
'a'	char 值	一个字符值	单字符判断和赋值
"a"	const char[2]	'a'、'\0'	一段以零结尾的文本
""	const char[1]	只有 '\0'	空文本

```
char ch = 'a';           // 合法
const char* p = "a";     // 合法：指向字面量首元素
char bad = "a";          // 错误示例：数组不能这样变成 char
```

给 `char` 初始化的是一个字符值，给 `p` 初始化的是经过数组转换得到的首元素指针。引号差异改变的是整个数据表示，不是简单的排版习惯。类型规则见 [字符字面量](#) 与 [字符串字面量](#)。

4. char 数组怎样保存文本？

```
char text[] = "abc";
```

`char` 是元素类型；`text` 是数组名；空的 `[]` 允许编译器由初始化内容推断数量；"abc" 使这个数组按字符内容初始化，并包括末尾 `\0`。`text` 的类型为 `char[4]`。

下标	元素内容	含义
0	'a'	第一个文本字符
1	'b'	第二个文本字符
2	'c'	第三个文本字符
3	'\0'	C 字符串结束标记

数组中不保存双引号。text[1] = 'X'; 修改数组中的第二个元素，文本变为 "aXc"。这里 text 是自己的可修改数组，并不是让一个指针直接指向不可修改的字面量对象。

5. \0 是什么？为什么不是字符 '0'？

'\0' 是数值为 0 的空字符。C 风格字符串用它标记文本结束。反斜杠开启转义写法，后面的 0 在这里表示零字符；它们合在一起描述一个字符值。

写法	是什么	是否代表 C 字符串终止标记
'\0'	值为 0 的 char	是
'0'	数字零的字符编码	否
0	整数字面量	赋给 char 时可得到零字符
nullptr	空指针字面量	否，是指针层次的概念

nullptr 讨论“指针当前有没有目标”；'\0' 讨论“字符数组中的这个元素是否为结束标记”。把两者混淆，就是把位置和位置里的数据混淆。

```
char a[] = {'a', '0', 'b', '\0'};
char b[] = {'a', '\0', 'b', '\0'};
```

a 的 C 字符串文本为 a0b，长度 3；b 的 C 字符串文本为 a，长度 1。b[2] 中的 'b' 仍然在数组里，只是不属于从 b[0] 开始、遇第一个零字符结束的那段 C 字符串。

6. 字符数组和 C 字符串怎样区分？

字符数组是存储容器；C 字符串是该存储中从某处开始、以零字符结束的一段字符序列。 不是所有 char 数组都自动构成合法 C 字符串。

```
char good[4] = {'a', 'b', 'c', '\0'};
char raw[3] = {'a', 'b', 'c'};
```

good 有结束标记，可作为这里的 C 字符串。raw 只是三个字符的数组，没有终止零。逐个按下标输出 raw[0]、raw[1]、raw[2] 是一件事；把 raw 交给要求零结尾的操作是另一件事，后者可能读过数组边界。

这就是“明明分配了数组，为什么还是越界”的常见根源：操作需要的停止条件没有在数组内出现。

7. 长度、容量与 sizeof：三个数字分别数什么？

```
char text[8] = "abc";
```

容量为 8 个 char 元素。初始化的前四个元素是 a、b、c、零字符，其余元素也被零初始化。以 text 开始的 C 字符串长度为 3，不包含终止零。sizeof(text) 为 8 字节，因为 sizeof(char) 为 1。

指标	本例数值	计数对象
数组容量	8	所有存储槽位
C 字符串长度	3	首个零字符之前的文本元素
当前字符串占用的必要槽位	4	三个字符加一个终止零
sizeof(text)	8	整个数组的字节数

容量为 C 的缓冲区若用于保存普通零结尾文本，最多容纳 C-1 个非零字符，还需一个槽位放 \0。不要把最后一个槽位也用完后忘记结束标记。

```
char exact[4] = "abc"; // 合法：正好含终止零
char bad[3] = "abc";   // C++ 错误示例：装不下终止零
```

8. const char* 是“不能移动的指针”吗？

```
const char* p = "abc";
++p;
```

const char* 是“指向 const char 的指针”。它限制通过 *p 修改目标字符，并不限制 p 保存的指针值。上例 p 可以从 a 的位置移到 b 的位置；*p = 'X'；不合法。

与之对照：

```
char text[] = "abc";
char* const q = text;
q[0] = 'X';           // 合法：目标数组可修改
// ++q;               // 错误示例：q 本身是 const 指针
```

从名字附近的声明结构读取：const char* p 让目标只读；char* const q 让指针对象本身不能改指向。二者都 const 的写法是 const char* const r。

普通字符串字面量不能被修改。C++17 下写 char* p = "abc"；本身不合法；不要用强制转换绕过后修改字面量。

9. 为什么 cout << p 会输出文本，不只是地址？

```
char text[] = "abc";
const char* p = text;
std::cout << p;
```

输出流对 char 指针有专门的文本输出操作，它从该位置读取字符直到遇到零字符。因此输出 `abc`。这不意味着 `p` 中保存了整个文本；`p` 仍然只保存指针值。

若要观察对象地址，可写：

```
std::cout << static_cast<const void*>(p);
```

`static_cast<const void*>` 在这里把对象指针转换为通用的只读对象地址类型，以使用地址输出形式。地址的具体字符表示由实现决定。理解这个例子只需知道“同一个指针值，输出操作可按类型选择不同行为”。M8 会进一步讲重载。

10. 怎样计算长度？停止条件在哪里？

已经保证 `s` 指向有效、零结尾字符序列时，可以逐个计数：

```
int length(const char* s) {
    int n = 0;
    while (s[n] != '\0') {
        ++n;
    }
    return n;
}
```

`const char* s` 接收首位置；`n` 表示当前下标，也表示已经数过多少个文本字符；`s[n]` 读取当前元素；不是零就计数并前进；遇到零时退出，返回 `n`。“`abc`”的下标依次检查 0、1、2、3，最后返回 3。

这里的前提非常关键：`s` 非空且能在有效范围内遇到零，长度在 `int` 可表示范围内。该教学函数没有长度参数，不能凭空知道任意缓冲区边界。标准库 `<cstring>` 的 `std::strlen` 也要求合法零结尾序列，返回类型为 `std::size_t`。

11. 复制字符内容，不是复制首地址

```
char source[] = "cat";
const char* p = source;
const char* q = p;
```

`q` 和 `p` 指向同一份 `source`，没有创建第二份文本。`source[0] = 'b'`；后，两者看到的都是 `bat`。**复制指针值与复制字符内容是两件事。**

要得到独立文本，需要另一个字符数组，逐元素写入，并复制终止零。例如小数组教学版：

```
char dest[4]{};
for (int i = 0; i < 4; ++i) {
    dest[i] = source[i];
}
```

本例的 4 包含终止零，且已知源和目的都至少有四个元素。通用函数必须额外处理容量和输入范围，不能照搬固定的 4。

内建数组不能在声明以后直接写 `dest = source`；来完成内容赋值。A1 把字符数组包装进 `str` 后，可以为 `str` 对象定义赋值操作；那是新类型提供的行为，不是内建数组突然支持赋值。

12. 字符串比较为什么不能只比较指针？

```
char a[] = "cat";
char b[] = "cat";
```

a 和 b 是两个不同数组，内容相同。若在进行比较中转换为首元素指针，指针相等比较讨论的是位置关系，不能代替逐字符内容比较。

`std::strcmp(a, b)` 比较零结尾文本内容：返回 0 表示内容相同；负值表示第一段文本在字典序上更小；正值表示更大。**不能假定非零结果固定等于 -1 或 1**。例如 "cat" 与 "car" 比到第三个字符才决定顺序，"ab" 与 "abc" 则由前者先结束决定顺序。

字典序是按首个不同字符或前缀终止位置比较，不是比较长度大小，也不是做数值大小比较：文本 "10" 在普通数字字符顺序下排在 "2" 前面。

13. 大小写转换先说明字符范围

只处理英文 ASCII 字母的课程题，可能用 'a' 到 'z' 的范围判断和编码差值转换；这种写法应注明编码前提。标准库 `<cctype>` 提供 `std::toupper`、`std::tolower` 等转换函数。

```
#include <cctype>
char ch = 'a';
char upper = static_cast<char>(
    std::toupper(static_cast<unsigned char>(ch))
);
```

先转换为 `unsigned char`，确保传入字符值在这些函数要求的范围内；函数返回 `int`，再转为 `char` 保存。转换还受当前字符分类环境影响。初学时先会理解“读一个字符、判断或转换、写入输出位置”，不要先把一串转换符当咒语背。

“返回大写副本”应先建立独立结果，再改结果的元素；“原地转换”才会修改输入。A1 的 `upper/lower` 接口属于前者。

14. `std::string` 是另一种类型，不是 `char*` 的别名

`std::string` 是标准库提供的字符串类，管理字符序列及相应存储，提供长度、拼接、复制等操作。使用前包含 `<string>`。

```
std::string s = "abc";
std::string t = s;
t[0] = 'X';
std::cout << s << ' ' << t << '\n';
```

输出 `abc Xbc`。t 是另一个 `string` 对象，修改 t 的内容不会同步修改 s。s.size() 返回存储序列长度；`s += "de"`；追加内容；`s == t` 按 `string` 的内容规则比较。

形式	自己管理文本长度与存储吗	常见长度取得方式
char 数组	只有固定元素存储，含义由程序规定	容量另记；C 文本可扫描零字符
char 指针	只保存位置，不自动管理文本	需知道它所指序列的规则
std::string	类型提供管理和操作	.size() 或 .length()

std::string 可以保存嵌入的零字符。例如 `std::string s("a\0b", 3)`；明确复制三个字符，size 为 3；若把其 `c_str()` 交给按零结尾读取的操作，从首位置观察的 C 字符串长度则是 1。

`c_str()` 提供只读的零结尾字符访问入口，不把 string 变成 char 数组。不要在 string 销毁或可能使指针失效的修改之后，继续依赖先前保存的 `c_str` 指针。类接口规则可查 [标准字符串类](#)。

15. 输入一个词和输入整行，不是同一个任务

```
std::string word;
std::cin >> word;
```

通常跳过前导空白，再读取一个词，遇空白停止。输入 `hello world`，word 得到 `hello`。若任务需要整行：

```
std::string line;
std::getline(std::cin, line);
```

它按行分隔符读取，保留行中的空格，取出分隔符但不把它存进 line。这里假定调用前输入位置就是目标行开头；若刚用 `>>` 读完数字，遗留换行可能让 `getline` 先得到空行，需按输入协议处理。

固定字符数组可以用 `std::cin.getline(buffer, capacity)` 明确容量；本讲采用 string 输入讲区别，不用 C++17 中没有给定宽度的 `>> buffer` 示范任意长度输入。

16. 读懂当前工作区 A1 的真实表示

当前 `assignment1/string.h` 中有：

```
const int maxlen = 1024;
struct str {
    char data[maxlen + 1] = {'\0'};
    // 这里省略头文件中的其他成员
};
```

这是一段摘录，省略标记不是原接口的一部分。`maxlen + 1` 给出 1025 个 char 槽位，额外一个位置用于终止零；不表示能保存 1025 个非零文本字符。数组初始化还会把其余元素零初始化，因此默认文本开始就是空的。

真实函数声明包括：

```
int str_len(const str& s);
void str_clear(str& s);
str upper(const str& src);
void str_copy(str& dest, const char* src);
```

逐个读：str_len 借用 s 读取并返回整数；str_clear 有权修改 s；upper 通过 const 引用读取输入，返回一个 str 对象作为结果；str_copy 可以改 dest，通过 const char* 读取零结尾源字符。

这些声明让 M2 的“谁能改谁”与本册的“文本怎样存”合在一起。当前公开自测还检查满容量截断和末尾零字符；讲义不据此补造其他未给出的教师要求。

17. 完整程序：长度、容量、独立复制同时观察

```
#include <cstring>
#include <iostream>
#include <string>

int main() {
    char text[8] = "abc";
    text[1] = 'X';
    std::string copy = text;
    text[0] = 'Z';
    std::cout << text << ' ' << copy << '\n';
    std::cout << std::strlen(text) << ' '
              << sizeof(text) << '\n';
}
```

先 text 为 aXc，再用其内容构造独立 string，随后 text 变成 ZXc。第一行输出 ZXc aXc；第二行输出 3 8。看见同一个文本初始化了新对象，并不表示后来仍共享同一份可修改内容。

回到 lecture notes：把字符串操作连成完整能力

A. 字符分类：检测一个字符，而非检测整个单词

L4 第 9 张列出 isdigit、isalpha、isalnum、islower、isupper、isspace，以及 toupper、tolower。前六项回答分类问题，后两项产生转换后的字符编码。标准 <cctype> 的这些函数通常接收并返回 int；分类结果按零/非零判断，课件中的 bool(char) 是便于阅读的接口简写。

```
char ch = '7';
bool digit = std::isdigit(static_cast<unsigned char>(ch)) != 0;
```

先将 char 转成 unsigned char 是为了让实参落在函数允许的字符值范围里。这里 digit 为 true，但 ch 仍是 '7'；检查并没有把它转换成整数 7。isspace 也不只检查普通空格，换行、制表等都属于常见空白。

课件用 ASCII 来讲 'a'=97、空格=32，适用于本册的普通 ASCII 示例；ASCII 基本集合为 128 个编码，不是 256 个。8 位字节能表示 256 种位模式，这是另一个计数。C++ 本身也不强制所有实现都采用 ASCII。

B. L4 第 24-26 张：+ 要逐段成立

"Hello, " << name 不是一般的字符串拼接表达式；cout 的输出链是流操作。真正的 + 需要检查左右类型：

```
std::string good1 = std::string("hello") + "," + "world";
std::string good2 = "hello" + std::string(",") + "world";
// 错误示例：最左边两个字面量不能相加
// std::string bad = "hello" + "," + std::string("world");
```

+ 左结合，第三行先尝试 "hello" + ","。它们是字符数组字面量，在相应表达式中转换为指针；C++ 没有两个指针的这种加法。后面存在 string 对象，也不能修好前面已经不成立的子表达式。前两行第一次 + 已涉及 string，所以得到新的 string，后续再追加文本。

相邻字面量的另一种写法 "hello" ", " "world" 会在翻译阶段合并，这是不同规则，不能拿来证明两个字面量用 + 可以相加。

C. L4 第 27、32、34 张：读字符串与改字符串

下面所有操作从 `std::string s="banana";` 开始理解。

操作	它做什么	本例结果或状态
<code>s.length()</code>	数字符串中的字符单位	6
<code>s.at(1)</code>	检查边界后访问第 2 个字符	'a'
<code>s.substr(1,3)</code>	从下标 1 复制最多 3 个字符	新 string 为 "ana", s 不变
<code>s.find("na",2)</code>	从下标 2 开始找首次匹配	2
<code>s.find('x')</code>	找字符 x	<code>std::string::npos</code>
<code>s.erase(1,2)</code>	从下标 1 原地删除两个字符	s 变 "bana"

`pos` 是起点下标，`count` 或 `len` 是数量，不能把 `s.substr(1,3)` 当成“下标 1 到 3 之外”的切片。`find` 找不到时，应与 `std::string::npos` 比较，不要把这个无符号特殊值随意当作可访问下标。

独立从 `s="banana"` 开始，`s.insert(1,"XY")` 得到 `"bXYanana"`；`s.replace(1,2,"X")` 则删除原下标 1 起两个字符并放入 X，得到 `"bXana"`。它们修改原对象，与返回新字符串的 `substr` 不同。

正常字符位置满足 `i < s.size()`。`at` 对越界会抛出 `std::out_of_range`；不要把课件自定义 `str::operator[]` 的抛异常示例，直接当成 `std::string::operator[]` 的行为。L4 第 23 张的自定义片段也未检查负下标，不能照抄为安全的通用接口。

D. L4 第 33、35-38 张：把单点操作组合起来

反转字符串要依次取得原来的倒数第一个、倒数第二个字符。下面是按课件任务改写的教学版，避免空字符串时先计算无符号 `length()-1`：

```
std::string reversed(const std::string& s) {
    std::string result;
    for (std::size_t i = s.size(); i > 0; --i) {
        result += s[i - 1];
    }
    return result;
}
```

`i` 表示尚未处理的前缀长度：初始等于 `size`；`i>0` 才访问 `i-1`；`i` 从 3、2、1 递减时，访问下标 2、1、0。输入为空时 `i=0`，循环一次也不执行，结果仍为空。输入 `"desserts"` 得到 `"stressed"`。

回文是正着和倒着相同的文本。课件任务可写成 `s == reversed(s)`；手推时也可以逐对比 `s[i]` 和 `s[n-1-i]`，只比较前半。一对不同就可判 `false`，全部对应相同才为 `true`。

L4 第 35-36 张的大写转换有两个不同契约：参数为 `string` 时修改的是独立参数副本，返回新文本；参数为 `string&` 时可以直接改原本。“in place 不安全”在这里应理解为可能破坏调用者期待的原内容，不能据此说所有原地操作都必然发生内存错误。

L4 第 38 张的 acronym 任务则要检测单词开头。为使规则具体，本册教学变式规定只用普通空格分词；当前字符不是空格，且位于下标 0 或其前一位是空格时，才是单词首字符。先辨认位置，再取字符；这能避免把每个非空格字符都收进去。

练习

题 1：数元素，不数引号

分别说明 'a'、"a"、"" 的类型或结构，并求 sizeof('a')、sizeof("a")、sizeof("")。

题 2：中间有零字符

```
char text[6] = {'a', 'b', '\\0', 'c', 'd', '\\0'};
```

容量是多少？从 text 开始的 C 字符串长度是多少？text[3] 是什么？输出 std::cout << text; 会显示什么？

题 3：复制指针还是复制内容？

```
char a[] = "cat";  
const char* p = a;  
std::string s = a;  
a[0] = 'b';
```

p 所指文本和 s 的内容分别是什么？

题 4：容量边界

容量为 5 的 char 数组，能保存多少个普通非零字符并保留一个终止零？char a[3] = "abc"; 在 C++17 中是否正确？

题 5：只读的是谁？

const char* p = "abc"; 后的 ++p 与 *p = 'X' 分别是否合法？若改为可修改数组上的 char* const q, 结论有什么变化？

题 6：A1 接口阅读

str upper(const str& src); 为什么同时出现返回类型 str 和参数 const str&？若希望修改原对象，当前声明能否通过 src 直接写 data？

练习详细解析

题 1 解析

普通单字符字面量 'a' 的类型是 char, sizeof 为 1; "a" 是 const char[2], sizeof 为 2; 空字符串 "" 是 const char[1], sizeof 为 1。后两者分别包括末尾零字符。引号属于源代码语法, 不是额外数组元素。

题 2 解析

容量 6; 扫描从 text[0] 开始, 数到 a、b, 在 text[2] 的零字符停止, 因此长度 2; text[3] 仍为 'c'。输出文本是 ab。错误答案 5 通常把“除最后一个零以外的数组槽位”都当成字符串, 忽略了第一次遇零就终止。

题 3 解析

p 保存 a 的首元素位置, 因此 a[0] 改为 b 后, p 看到 bat。s 在构造时复制了内容, 仍为 cat。关键判断不是“谁先由 a 初始化”, 而是初始化的是位置值, 还是一个管理独立字符序列的新对象。

题 4 解析

最多 4 个非零字符, 另留一个槽位放零。char a[3] = "abc"; 在 C++17 中不合法, 初始化所需四个元素装不下。不要从别的语言或 C 的特殊初始化规则推断本题。

题 5 解析

const char* 限制目标访问: ++p 可移动到字面量中下一个字符位置, *p = 'X' 不合法。char* const q 限制指针对象: 不能 ++q, 但若其目标是有效的可修改数组, 可通过 *q 改字符。先确定 const 限制的是哪一层。

题 6 解析

返回 str 表示函数产生一个结果对象; const str& 表示通过只读引用借用输入, 不为参数本身创建独立的 str 副本。函数不能通过 src 直接修改普通 data 成员; 可创建自己的结果并转换结果内容。当前头文件注释也规定返回转换后的副本。

复习时抓住这条线

字符值 → 字符数组 → 零结束规则 → 长度与容量 → 指针只保存位置 → 类可以管理内容。下一册将解释“自己定义一种保存 data 的类型”意味着什么, 以及构造和赋值为什么不是一回事。

课件与旧卷训练：题面和详细解析

S1. E22 Q3（第 2 页）：两个声明会得到相同文本吗？

原代码：

```
std::string str1 = "hello,world";
std::string str2 = "hello" + "," + std::string("world");
```

答案 **False**。第一句可行；第二句先分组为 ("hello"+"")+std::string("world")，最内层没有合适的 +，所以声明不能通过编译。这不是“输出内容不同”，而是第二个对象不能按这段合法代码创建。

最小修正可把第一项改为 std::string("hello")，或把中间项改为 std::string(",")。不要只把最后一项转换成 string，因为失败发生在到达最后一项之前。

S2. L4 第 15 张：sizeof、strlen 与越界

课件例题核心代码，省略头文件与主函数外壳：

```
char cstr[] = "hello";
cout << cstr << endl << sizeof cstr << endl << strlen(cstr) << endl;
strcpy(cstr, "hello world");
cout << cstr << endl << sizeof cstr << endl << strlen(cstr) << endl;
```

第一组输出动作对应 "hello"、6、5。数组由五个可见字符和一个终止零初始化，容量为 6；strlen 在首个零之前数得 5。

下一次 strcpy 需要写入 11 个可见字符与一个终止零，共 12 个槽位，目的只有 6 个，导致越界写和未定义行为。**整个程序没有可保证的完整输出；“第一组为 hello/6/5”只是到达错误操作前的局部语义分析。** 不能为后半段指定文本、长度或报错方式，也不能用某次运行来验证非法访问。

S3. L6 第 41、44 张：紧凑循环为什么能停止？

课件给出 while (*str++ != '\0') { ++n; }。假定 str 指向有效的零结尾序列：每轮用旧位置读取一个字符，再推进 str；读到零时条件为假，n 不再增加。所以 "cat" 返回 3，而 str 最终已经越过终止零的位置。

L6 第 44 张的 while (*dst++ = *src++); 还把赋值结果用于条件。a、b、c 各被复制一次，条件非零；终止零也先被复制，再让条件变假。循环体是末尾分号所表示的空语句。它不自动检查目的容量；必须先有足够空间、有效源、正确的非重叠范围等前提。

这两个表达式放在 M3-M5 之后才讲：先懂旧指针值与目标访问，再看循环停止条件。A1 自己有截断契约，不能把这个无边界的紧凑 strcpy 原样当成通用解法。

S4. L4 第 37-38 张教学变式

判断 "level" 与 "lever" 是否回文；按上文“普通空格分词”规则，从 "not in my back yard" 取首字母；从全新 string s="banana" 计算 s.substr(2,2) 与 s.find("na",3)。

解析。 level 的对应字符 l/l、e/e 都相同，中间 v 无须再配对，所以 true。lever 第一对 l/r 已不同，所以 false。缩写规则跳过开头及重复空格，只取 n、i、m、b、y，结果 "nimby"。substr 从下标 2 取两个字符，为 "na"；find 从下标 3 才开始，跳过位于 2 的匹配，下一次匹配在 4，所以返回 4。若求不到匹配才返回 npos。

S5. A1 第 1-2 页：用边界例子检验理解

题意转述：假设老师把 `maxlen` 改成 3，复制输入 "ABCDE" 应得到什么？`upper` 是否修改输入？`+`、`+=`、流输入分别有什么行为要求？

解析。 保留前三个字符 ABC，并在 `data[3]` 写终止零；容量仍要有 `maxlen+1` 个元素，不能只给三个槽位。
`upper` 返回改变英文小写字母后的新 `str`，输入保持原状；`+` 返回新结果并保持操作数；`+=` 修改左对象且返回左对象引用；`>>` 读一个由空白分隔的词，过长词只保存前 `maxlen` 个字符并丢弃其余部分。`<<` 输出内容且不额外添加换行。

这些是题面行为契约，用来读懂和自行检查实现，不在此提供可提交的完整 `string.cpp`。标准 `string` 方法在一般教学中有用，但 A1 明确禁止用 `std::string`、`getline` 或所列字符串库函数代替逐字符实现。